Virtual Programming Fundamentals Internship
in Ahmedabad

Content Development and Curriculum Building

A programming fundamentals module for non-programmers

Author: Meghana Krishnakumar Nair
Week 2 of 4 | Offer received 2 November 2026
Internship period: 2 November 2026 to 30 November 2026
Report prepared on 6 November 2026

This curriculum turns the week 1 plan into a week 2 deliverable: a complete curriculum with lesson plans, interactive activities, executed code examples, and an evidence-backed design for teaching programming fundamentals to non-programmers.

Table of contents

Use the list below to find each section quickly. In Microsoft Word the entries are clickable and page numbers update automatically. In this file the contents are a live field, so right-click the list and choose Update Field after opening in Word to refresh the numbers.

Right-click and choose Update Field in Word to populate page numbers after opening.

Curriculum content development and lesson plans ...
1 About this curriculum ... 5
2 Research summary ... 8
3 Curriculum architecture ... 11
4 Session-by-session plan ... 14
5 Lesson plan: m1 ... 18
6 Lesson plan: m2 ... 22
7 Lesson plan: m3 ... 25
8 Lesson plan: m4 ... 28
9 Lesson plan: m5 ... 31
10 Lesson plan: m6 ... 34
11 Lesson plan: m7 ... 37
12 Lesson plan: m8 ... 40
13 Lesson plan: m9 ... 43
14 Lesson plan: m10 ... 46
15 Assessment strategy ... 50
16 Supporting non-programmers ... 53
17 Review and refinement log ... 56
18 Validation plan and known limitations ... 59
Appendix A: activity catalogue ... 62
Appendix B: quiz bank ... 65
Appendix C: error catalogue ... 68
Appendix D: glossary ... 71
Appendix E: readability and audit results ... 73
Appendix F: reference list ... 75
Appendix G: reproducibility ... 77

Executive summary

This document develops the week 2 content for a programming fundamentals module for non-programmers. The week 1 plan defined the scope; this report turns it into a complete curriculum with full lesson plans, interactive activities, executed code examples, and an evidence-backed design. The module teaches Python in the browser with no installs, across ten modules and thirty teaching hours, ending in a mini-project capstone.

Every technical concept is explained in plain English first, then tied to a real-life analogy that states where it breaks down. Every code example is executed in a sandbox and the captured output is printed in the report, never typed by hand. Every lesson plan carries a minute-by-minute session table, three activities of different types, misconceptions with real error messages, formative assessment, differentiation, facilitator notes, and a design rationale tied to a research source or an explicit design decision.

The final deliverable is a Word document that presents the complete curriculum, with lesson plans, topics, learning activities, annotations, examples, and an interactive activity set. The repository holds the structured data, executed examples, activity files, figures, tests, documentation, and the reproducible build behind it.

1 About this curriculum

This internship offer was received on 2 November 2026 and the internship runs from 2 November 2026 to 30 November 2026. This week 2 report was prepared on 6 November 2026, five days after the week 1 report. It develops the content and curriculum building task: the aim is to introduce the fundamentals of programming to non-programmers with practical examples that are easy to grasp.

The learners are adults or students who have never written a line of code. Their prior knowledge is everyday computer use only. The curriculum assumes no programming background, no installs, and a browser. It does not assume any maths beyond counting.

By the end of the module, a learner will be able to: describe a task as steps, store information in variables, ask and show through input and output, choose with conditionals, repeat with loops, bundle steps into functions, organise values into lists and dictionaries, read and write files, read a traceback, and combine the taught concepts into one working small program they can explain in plain words.

2 Research summary and evidence map

Good planning starts with honest evidence. Public beginner courses and teaching studies point in the same direction: the ideas in programming are not especially hard, but the way we introduce them can make them feel hard. Learners struggle with unfamiliar words, with holding many steps in mind at once, with mistaking a variable for a box, and with the emotional sting of an error message.

This curriculum therefore sequences ideas carefully, uses worked examples and ordering tasks before free writing, makes errors normal, pairs learners, gives quick feedback, and ends with a useful personal project. The design decisions in the lesson plans are traceable to sources in the week 2 research register, or are explicitly labelled design decisions.

3 Curriculum architecture

The curriculum is ten modules totalling thirty teaching hours, built from the week 1 module list and kept consistent with it.

module	hours	minutes	prerequisites	concepts introduced
m1	3	180	none	computational thinking, algorithm, sequence, decomposition, pseudocode
m2	3	180	m1	variable, assignment, string, integer, float, boolean
m3	2	120	m2	input, print, prompt, display
m4	3	180	m3	if, else, boolean expression, comparison operator
m5	3	180	m4	while, for, range, iteration
m6	3	180	m5	def, return, parameter, argument, function call
m7	3	180	m6	list, index, append, dictionary, key, value
m8	3	180	m7	open, read, write, with statement
m9	3	180	m1-m8	syntax error, runtime error, logic error, traceback
m10	4	240	m1-m9	capstone: applies all prior concepts

Hours add up to 30 (1800 minutes). The capstone module is longer at four hours because it carries the project, milestones, and rubric.

4 Session-by-session plan

The week 2 plan keeps the week 1 delivery shape: a ninety-minute session of arrive and recall, worked example, pair task or Parsons problem, predict-run-modify, and exit ticket, plus a buffer. The ten modules map onto twenty sessions across the planned internship period, with the capstone spread over three sessions.

session	module	phase	planned date	duration (min)	blocks
s1	m1	content	2026-11-07 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s2	m1	content	2026-11-08 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s3	m2	content	2026-11-09 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s4	m2	content	2026-11-10 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s5	m3	content	2026-11-11 (planned)	120	10 + 20 + 25 + 20 + 10 + 35
s6	m4	content	2026-11-12 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s7	m4	content	2026-11-13 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s8	m5	content	2026-11-14 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s9	m5	content	2026-11-15 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s10	m6	content	2026-11-16 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s11	m6	content	2026-11-17 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s12	m7	content	2026-11-18 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s13	m7	content	2026-11-19 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s14	m8	content	2026-11-20 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s15	m8	content	2026-11-21 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s16	m9	content	2026-11-22 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s17	m9	content	2026-11-23 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s18	m10	capstone	2026-11-24 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s19	m10	capstone	2026-11-25 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s20	m10	capstone	2026-11-26 (planned)	60	10 + 20 + 15 + 5 + 10

All dates in the session table are planned and rise from the baseline of 5 November 2026. Weekends and the 25 November 2026 holiday are skipped, consistent with week 1.

1 About this curriculum

This internship offer was received on 2 November 2026 and the internship runs from 2 November 2026 to 30 November 2026. This week 2 report was prepared on 6 November 2026, five days after the week 1 report. It develops the content and curriculum building task: the aim is to introduce the fundamentals of programming to non-programmers with practical examples that are easy to grasp.

The learners are adults or students who have never written a line of code. Their prior knowledge is everyday computer use only. The curriculum assumes no programming background, no installs, and a browser. It does not assume any maths beyond counting.

By the end of the module, a learner will be able to: describe a task as steps, store information in variables, ask and show through input and output, choose with conditionals, repeat with loops, bundle steps into functions, organise values into lists and dictionaries, read and write files, read a traceback, and combine the taught concepts into one working small program they can explain in plain words.

2 Research summary and evidence map

Good planning starts with honest evidence. Public beginner courses and teaching studies point in the same direction: the ideas in programming are not especially hard, but the way we introduce them can make them feel hard. Learners struggle with unfamiliar words, with holding many steps in mind at once, with mistaking a variable for a box, and with the emotional sting of an error message.

This curriculum therefore sequences ideas carefully, uses worked examples and ordering tasks before free writing, makes errors normal, pairs learners, gives quick feedback, and ends with a useful personal project. The design decisions in the lesson plans are traceable to sources in the week 2 research register, or are explicitly labelled design decisions.

3 Curriculum architecture

The curriculum is ten modules totalling thirty teaching hours, built from the week 1 module list and kept consistent with it.

module	hours	minutes	prerequisites	concepts introduced
m1	3	180	none	computational thinking, algorithm, sequence, decomposition, pseudocode
m2	3	180	m1	variable, assignment, string, integer, float, boolean
m3	2	120	m2	input, print, prompt, display
m4	3	180	m3	if, else, boolean expression, comparison operator
m5	3	180	m4	while, for, range, iteration
m6	3	180	m5	def, return, parameter, argument, function call
m7	3	180	m6	list, index, append, dictionary, key, value
m8	3	180	m7	open, read, write, with statement
m9	3	180	m1-m8	syntax error, runtime error, logic error, traceback
m10	4	240	m1-m9	capstone: applies all prior concepts

Hours add up to 30 (1800 minutes). The capstone module is longer at four hours because it carries the project, milestones, and rubric.

4 Session-by-session plan

The week 2 plan keeps the week 1 delivery shape: a ninety-minute session of arrive and recall, worked example, pair task or Parsons problem, predict-run-modify, and exit ticket, plus a buffer. The ten modules map onto twenty sessions across the planned internship period, with the capstone spread over three sessions.

session	module	phase	planned date	duration (min)	blocks
s1	m1	content	2026-11-07 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s2	m1	content	2026-11-08 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s3	m2	content	2026-11-09 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s4	m2	content	2026-11-10 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s5	m3	content	2026-11-11 (planned)	120	10 + 20 + 25 + 20 + 10 + 35
s6	m4	content	2026-11-12 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s7	m4	content	2026-11-13 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s8	m5	content	2026-11-14 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s9	m5	content	2026-11-15 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s10	m6	content	2026-11-16 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s11	m6	content	2026-11-17 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s12	m7	content	2026-11-18 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s13	m7	content	2026-11-19 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s14	m8	content	2026-11-20 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s15	m8	content	2026-11-21 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s16	m9	content	2026-11-22 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s17	m9	content	2026-11-23 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s18	m10	capstone	2026-11-24 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s19	m10	capstone	2026-11-25 (planned)	90	10 + 20 + 25 + 20 + 10 + 5
s20	m10	capstone	2026-11-26 (planned)	60	10 + 20 + 15 + 5 + 10

All dates in the session table are planned and rise from the baseline of 5 November 2026. Weekends and the 25 November 2026 holiday are skipped, consistent with week 1.

m1: computational thinking and problem decomposition

module m1 of 10 - computational thinking and problem decomposition. duration 180 minutes. prerequisites: none.

objectives: m1-o1, m1-o2.

plain english and the real-life analogy

before any code, learners describe a task as a clear list of steps in plain words.

real-life analogy: giving directions to a friend who has never seen the street: write the steps in order, one after another, no skipping.

where this analogy breaks down: the friend might read faster than you meant; in code the computer cannot improvise..

worked example: a complete runnable program

```
steps = [
    "open the diary",
    "turn to page 3",
    "read the entry",
    "close the diary",
]
print("recipe made of " + str(len(steps)) + " steps")
```

line-by-line annotation: line 1: we call the list of steps a variable called steps. line 4: each line is one instruction, in order, like a recipe.

captured real output (executed in a sandbox, python 3.13):

```
recipe made of 4 steps
```

a second example that changes one thing

```
steps = [
    "boil water",
    "add tea bag",
    "wait four minutes",
    "add milk",
]
print("recipe made of " + str(len(steps)) + " steps")
```

captured real output for this variant:

```
recipe made of 4 steps
```

figure for the hardest idea in this module

Figure 1: curriculum map - modules, hours and dependencies

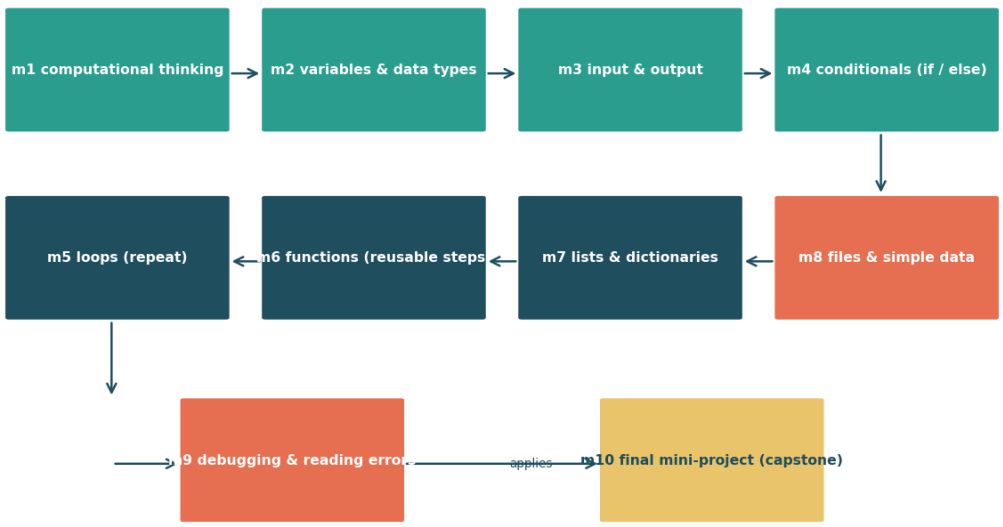


Figure 1: modules, hours and dependencies. *the key idea, stated as the takeaway.*

Image description: diagram for m1: curriculum map: modules, hours and dependencies

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	parsons problem	order scrambled lines	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m1-p1 (type parsons), serves objectives m1-o1, time budget 25 minutes.

instructions in plain words: read the plain-English steps and put them in the right order in the code.

starter code: steps = []

expected result / success criteria: the printed count equals 4 and the steps read in order.

hint ladder:

hint 1: think about what happens first.

hint 2: the print uses len(steps), the number of steps.

hint 3: order matters: sequence is the first idea.

activity m1-prm (type predict-run-modify), serves objectives m1-o1, m1-o2, time budget 20 minutes.

instructions in plain words: predict the output, run it, then change one thing.

starter code: steps = [
    "open the diary",
]

expected result / success criteria: output is the recipe line; changing order changes nothing visible here, so change the count.

hint ladder:

hint 1: predict before running.

hint 2: len() counts items.

hint 3: sequence means order.

activity m1-a1 (type pair_task), serves objectives m1-o1, time budget 25 minutes.

instructions in plain words: two learners order steps together on paper.

starter code: paper list

expected result / success criteria: both learners can state the steps in order.

hint ladder:

hint 1: one reads, one writes.

hint 2: talk first, type second.

common misconceptions

misconception m1-m1: a program is a paragraph of prose

real error or wrong output: no code executed; wrong mental model

facilitator response: show a recipe card vs a recipe sentence; code is instruction-like.

formative assessment

quiz m1-q1: what does a computer program do?

correct answer: follows a list of instructions in order

wrong option rationale: guesses what you mean - it detects the misconception that the program guesses.

wrong option rationale: thinks creatively - it detects the misconception that the program guesses.

objective tested: m1-o1, bloom level remember.

quiz m1-q2: breaking a task into smaller steps is called

correct answer: decomposition

wrong option rationale: debugging - it detects the misconception that decomposition is debugging.

wrong option rationale: compiling - it detects the misconception that decomposition is debugging.

objective tested: m1-o1, bloom level understand.

exit ticket m1-et1: in your own words, what is an algorithm? (objective m1-o1).

differentiation

learners who are stuck: give the m1-p1 starter with three steps already ordered and ask the learner to add one more.

learners who finish early: ask the learner to write steps for making a cup of tea and run it..

facilitator notes

pacing checkpoints: at minute 20 check that all pairs have an ordered set.

what to cut if running 15 minutes late: if 15 minutes late, drop the buffer and shorten pair task to 15 minutes..

setup checklist: open the colab link, confirm python runs, prepare scrambled lines..

why it is designed this way

the worked example gives a success within minutes, which sustains motivation [R9].

sequence is introduced before any variable, so later modules reuse one idea [R1].

m2: variables and data types

module m2 of 10 - variables and data types. duration 180 minutes. prerequisites: m1.

objectives: m2-o1, m2-o2.

plain english and the real-life analogy

a variable is a name for a value the program can read and change, like a labelled box.

real-life analogy: a labelled box where you can put a number or a word, and later take the label off and use a different value.

where this analogy breaks down: the box is not a box; reassigning changes what the name points to, not the box..

worked example: a complete runnable program

```
name = "meghana"
age = 26
height_cm = 165.5
print(name)
print(age)
print(height_cm)
```

line-by-line annotation: line 1: name holds the text meghana. line 2: age holds the whole number 26.

captured real output (executed in a sandbox, python 3.13):

```
meghana
26
165.5
```

a second example that changes one thing

```
name = "arjun"
age = 19
print("hello " + name + ", you are " + str(age) + " years old")
```

captured real output for this variant:

```
hello arjun, you are 19 years old
```

figure for the hardest idea in this module

Figure 5: variable as a labelled box - before and after assignment

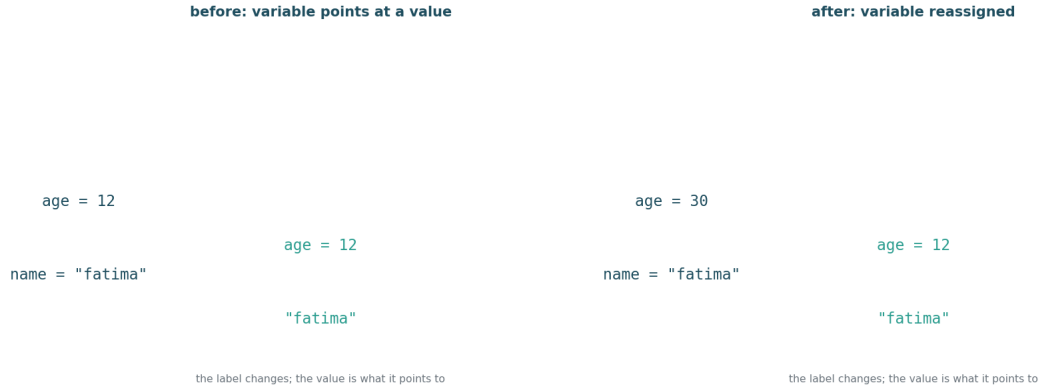


Figure 1: before and after assignment. *the key idea, stated as the takeaway.*

Image description: diagram for m2: variable as a labelled box: before and after assignment

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	debug-this	find the real bug	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m2-mis (type debug-this), serves objectives m2-o1, time budget 25 minutes.

instructions in plain words: find the real bug in almost-correct code.

starter code: name = "meghana"
age = 26
print(name)
print(agex)

expected result / success criteria: the code runs and prints meghana and 26.

hint ladder:

hint 1: run it and read the traceback.

hint 2: the last line names the bad token.

hint 3: did you mean age?

activity m2-pair (type pair_task), serves objectives m2-o2, time budget 25 minutes.

instructions in plain words: talk through the prediction before typing.

starter code: age = 12

expected result / success criteria: pair predicts 12 before running.

hint ladder:

hint 1: say the answer out loud.

hint 2: predict in words first.

activity m2-prm (type predict-run-modify), serves objectives m2-o1, m2-o2, time budget 20 minutes.

instructions in plain words: predict the output, run it, modify one thing.

starter code: age = 12
print(age)

expected result / success criteria: output is 12.

hint ladder:

hint 1: what is 12 plus nothing?

hint 2: just print age.

common misconceptions

misconception m2-m1: the variable is a box that holds the value

real error or wrong output: a reassign creates a new value; the name points at it

facilitator response: use the labelled-box story and show the before/after diagram.

formative assessment

quiz m2-q1: what does a variable hold?

correct answer: a value that can change

wrong option rationale: always the same number - it detects the misconception that variable is a box.

wrong option rationale: a box - it detects the misconception that variable is a box.

objective tested: m2-o1, bloom level apply.

quiz m2-q2: what does print do?

correct answer: shows a value to the user

wrong option rationale: stores a value - it detects the misconception that print stores.

wrong option rationale: fixes a bug - it detects the misconception that print stores.

objective tested: m2-o1, bloom level understand.

exit ticket m2-et1: what will this print: age = 3 then print(age)? (objective m2-o1).

differentiation

learners who are stuck: use the m2-mis starter and walk the traceback together.

learners who finish early: ask the learner to add a second variable and print both..

facilitator notes

pacing checkpoints: check prediction accuracy at exit ticket.

what to cut if running 15 minutes late: if 15 minutes late, shorten worked example to 12 minutes..

setup checklist: open colab, type the example live..

why it is designed this way

the worked example reduces cognitive load for the first variable lesson [R1][R5].

predict before run makes the variable misconception visible [R8].

m3: input and output

module m3 of 10 - input and output. duration 120 minutes. prerequisites: m2.

objectives: m3-o1.

plain english and the real-life analogy

a program talks to the person through input and output: ask, remember, show.

real-life analogy: ordering food at a restaurant: you say what you want (input), the kitchen writes it down (variable), the waiter brings the dish (output).

where this analogy breaks down: the kitchen cannot answer follow-up questions; it just follows the written ticket..

worked example: a complete runnable program

```
name = input("what is your name? ")
age = input("how old are you? ")
print("hello " + name + ". you will be " + str(int(age) + 1) + " next year.")
```

line-by-line annotation: line 1: input asks and remembers. line 2: int() turns the text into a number.

captured real output (executed in a sandbox, python 3.13):

```
hello meghana. you will be 27 next year
```

a second example that changes one thing

```
city = input("which city are you in? ")
print("noted: " + city)
```

captured real output for this variant:

```
noted: ahmedabad
```

figure for the hardest idea in this module

Figure 10: file read / write flow

files are read into variables, processed, then written back

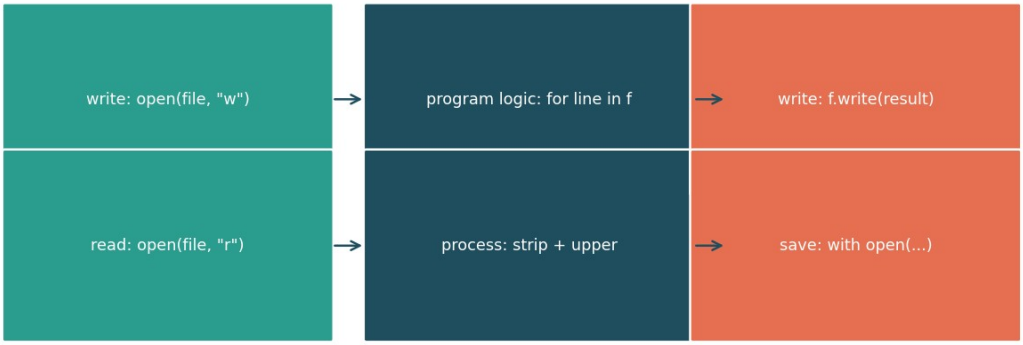


Figure 1: file read / write flow. *the key idea, stated as the takeaway.*

Image description: diagram for m3: file read / write flow

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	pair task	talk through prediction	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-120	buffer	buffer / recap	extra practice / recap	starter + colab

the blocks above sum to 120 minutes, which equals the module duration.

activities

activity m3-prm (type predict-run-modify), serves objectives m3-o1, time budget 20 minutes.

instructions in plain words: predict the output, run it, modify one thing.

starter code: name = input("what is your name? ")
print("hi " + name)

expected result / success criteria: output greets the typed name.

hint ladder:

hint 1: input returns text.

hint 2: plus joins strings.

activity m3-a2 (type pair_task), serves objectives m3-o1, time budget 25 minutes.

instructions in plain words: talk through the prediction before typing.

starter code: city = input("which city? ")

expected result / success criteria: pair predicts the note.

hint ladder:

hint 1: what will be printed?

hint 2: input returns text.

activity m3-a3 (type debug-this), serves objectives m3-o1, time budget 25 minutes.

instructions in plain words: find the real bug.

starter code: age = input("how old? ")
print("next year you will be " + str(int(age)) + " years old")

expected result / success criteria: works when input is a number.

hint ladder:

hint 1: what does int() do?

hint 2: what if the answer is a name?

common misconceptions

misconception m3-m1: input gives a number automatically

real error or wrong output: input always gives text; int() converts

facilitator response: show input then int() side by side.

formative assessment

quiz m3-q1: what does input do?

correct answer: asks the user and returns the typed text

wrong option rationale: gives a number - it detects the misconception that input gives a number.

wrong option rationale: saves to a file - it detects the misconception that input gives a number.

objective tested: m3-o1, bloom level understand.

quiz m3-q2: what does print do?

correct answer: shows text to the user

wrong option rationale: reads from a file - it detects the misconception that print reads.

wrong option rationale: sends an email - it detects the misconception that print reads.

objective tested: m3-o1, bloom level understand.

exit ticket m3-et1: what will this print: name = input(); print(name)? (objective m3-o1).

differentiation

learners who are stuck: run the m3-a3 starter with a numeric answer.

learners who finish early: ask the learner to add a second question and print both answers..

facilitator notes

pacing checkpoints: check that input was used.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, test input flow..

why it is designed this way

a quick win through output early sustains motivation [R9].

input/print are introduced together so the habit forms fast.

m4: conditionals (if / else)

module m4 of 10 - conditionals (if / else). duration 180 minutes. prerequisites: m3.

objectives: m4-o1.

plain english and the real-life analogy

a program can choose between two or more paths based on a condition.

real-life analogy: a traffic roundabout: if the light is green, go; if red, stop; otherwise follow the sign.

where this analogy breaks down: a roundabout has many roads; an if/else has a binary choice and a condition you must write precisely..

worked example: a complete runnable program

```
age = 12
if age >= 13:
    print("you can watch the scary movie")
else:
    print("you must watch the gentle cartoon")
```

line-by-line annotation: line 2: the condition is checked. line 3 and 4: each branch is a step.

captured real output (executed in a sandbox, python 3.13):

```
you must watch the gentle cartoon
```

a second example that changes one thing

```
name = "fatima"
if name == "fatima":
    print("welcome back, teacher")
else:
    print("hello, stranger")
```

captured real output for this variant:

```
welcome back, teacher
```

figure for the hardest idea in this module

Figure 6: decision flowchart - should the learner watch the scary movie?

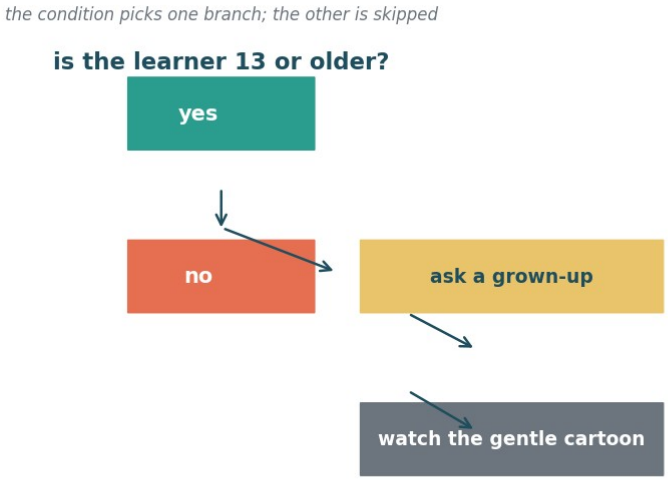


Figure 1: should the learner watch the scary movie. *the key idea, stated as the takeaway.*

Image description: diagram for m4: decision flowchart: should the learner watch the scary movie

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	parsons problem	order if / else lines	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m4-p1 (type parsons), serves objectives m4-o1, time budget 25 minutes.

instructions in plain words: order the if / else lines.

starter code: age = 12
if age >= 13:

```
    print("scary")
else:
    print("gentle")
```

expected result / success criteria: the condition is first and else pairs correctly.

hint ladder:

hint 1: condition goes first.

hint 2: else pairs with if.

activity m4-prm (type predict-run-modify), serves objectives m4-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code:

```
age = 12
if age >= 13:
    print("scary")
```

expected result / success criteria: prints gentle cartoon.

hint ladder:

hint 1: 12 is not 13.

hint 2: change the number.

activity m4-a2 (type pair_task), serves objectives m4-o1, time budget 25 minutes.

instructions in plain words: talk through the branch.

starter code: age = 12

expected result / success criteria: pair states the branch.

hint ladder:

hint 1: what is 12 >= 13?

hint 2: false means else.

common misconceptions

misconception m4-m1: the computer reads the story we tell it

real error or wrong output: the computer runs only the taken branch

facilitator response: trace which branch runs with a real run.

misconception m4-m2: if means always

real error or wrong output: if is a condition, not a constant

facilitator response: show an if that is always true and one that is false.

formative assessment

quiz m4-q1: what does an if check?

correct answer: whether a condition is true

wrong option rationale: whether a variable exists - it detects the misconception that if checks existence.

wrong option rationale: whether a file exists - it detects the misconception that if checks existence.

objective tested: m4-o1, bloom level apply.

quiz m4-q2: what does else mean?

correct answer: the other branch when the if is not true

wrong option rationale: always run this - it detects the misconception that else always runs.

wrong option rationale: stop the program - it detects the misconception that else always runs.

objective tested: m4-o1, bloom level apply.

exit ticket m4-et1: in words, when should the gentle cartoon play? (objective m4-o1).

differentiation

learners who are stuck: use the m4-p1 scrambled lines and sort them together.

learners who finish early: ask the learner to add an elif for a second age group..

facilitator notes

pacing checkpoints: check condition before branches.

what to cut if running 15 minutes late: if 15 minutes late, drop predict-run-modify to 10 minutes..

setup checklist: open colab, type example live..

why it is designed this way

conditions are the first real logic; worked example then pair task [R1].

prediction before run exposes condition misconceptions [R8].

m5: loops (repeat)

module m5 of 10 - loops (repeat). duration 180 minutes. prerequisites: m4.

objectives: m5-o1.

plain english and the real-life analogy

a loop repeats a block of steps until a condition becomes false, or for a known number of times.

real-life analogy: asking a child to count from one to ten by pointing at each number on a number line, one at a time, ten times.

where this analogy breaks down: the number line is physical; a for loop is a counter, and the stop condition must be written, not assumed..

worked example: a complete runnable program

```
total = 0
for i in range(3):
    total = total + 1
print(total)
```

line-by-line annotation: line 1: total starts at zero. line 2: loop runs three times. line 3: add one each time.

captured real output (executed in a sandbox, python 3.13):

```
3
```

a second example that changes one thing

```

```

```
count = 0
while count < 3:
    count = count + 1
print("done counting to " + str(count))
```

captured real output for this variant:

```
done counting to 3
```

figure for the hardest idea in this module

Figure 7: loop trace - variable values step by step

```
start |   total = 0   |   i = -   |   check stop: 0 < 3 -> true
```

```
  1   |   total = 1   |   i = 0   |   body: total = total + 1
```

```
  2   |   total = 2   |   i = 1   |   body: total = total + 1
```

```
  3   |   total = 3   |   i = 2   |   body: total = total + 1
```

```
 end  |   total = 3   |   i = -   |   stop: 3 < 3 -> false
```

Figure 1: variable values step by step. *the key idea, stated as the takeaway.*

Image description: diagram for m5: loop trace: variable values step by step

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	parsons problem	order loop lines	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m5-p1 (type parsons), serves objectives m5-o1, time budget 25 minutes.

instructions in plain words: order the loop lines so the stop condition works.

starter code: total = 0
for i in range(3):
    total = total + 1
print(total)

expected result / success criteria: output is 3.

hint ladder:

hint 1: range gives the count.

hint 2: the stop condition must end the loop.

activity m5-prm (type predict-run-modify), serves objectives m5-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code: total = 0
for i in range(3):
    total = total + 1
print(total)

expected result / success criteria: output is 3.

hint ladder:

hint 1: how many times?

hint 2: what does range(3) mean?

activity m5-a2 (type pair_task), serves objectives m5-o1, time budget 25 minutes.

instructions in plain words: talk through the loop.

starter code: total = 0

expected result / success criteria: pair states the stop condition.

hint ladder:

hint 1: what changes each round?

hint 2: when does it stop?

common misconceptions

misconception m5-m1: a loop means repeat forever

real error or wrong output: a loop needs a stop condition

facilitator response: show a while loop that never ends and how to break.

misconception m5-m2: the loop variable is the value

real error or wrong output: i is a counter, not the data

facilitator response: show with a list.

formative assessment

quiz m5-q1: what does range(3) give?

correct answer: three loop runs

wrong option rationale: three values - it detects the misconception that range gives values.

wrong option rationale: three numbers to add - it detects the misconception that range gives values.

objective tested: m5-o1, bloom level apply.

quiz m5-q2: when does a while loop stop?

correct answer: when the condition becomes false

wrong option rationale: after ten runs - it detects the misconception that while has a default stop.

wrong option rationale: never - it detects the misconception that while has a default stop.

objective tested: m5-o1, bloom level apply.

exit ticket m5-et1: how many times will this loop run: for i in range(2)? (objective m5-o1).

differentiation

learners who are stuck: use the m5-p1 scrambled lines and sort them.

learners who finish early: ask the learner to change range(3) to range(5) and predict the new total..

facilitator notes

pacing checkpoints: check stop condition each session.

what to cut if running 15 minutes late: if 15 minutes late, drop the worked example to 12 minutes..

setup checklist: open colab, type example live..

why it is designed this way

tracing is how novices read loops; loop trace figure supports this [R7].

parsons problems lower load for ordering loops [R2][R3].

m6: functions (reusable steps)

module m6 of 10 - functions (reusable steps). duration 180 minutes. prerequisites: m5.

objectives: m6-o1.

plain english and the real-life analogy

a function is a named, reusable bundle of steps that takes inputs and gives back one output.

real-life analogy: a vending machine: you press a button (call), the machine follows its internal steps (function body), and gives you a snack (return).

where this analogy breaks down: a vending machine cannot take its own button as input; functions take parameters and return one value, not many..

worked example: a complete runnable program

```
def greet(name):
    return "hello " + name + ", welcome back"
print(greet("fatima"))
```

line-by-line annotation: line 1: def makes a function called greet with a name parameter. line 2: return sends back the greeting.

captured real output (executed in a sandbox, python 3.13):

```
hello fatima, welcome back
```

a second example that changes one thing

```
def add(a, b):
    return a + b
print(add(2, 3))
```

captured real output for this variant:

```
5
```

figure for the hardest idea in this module

Figure 8: function as a machine - inputs, return value and call

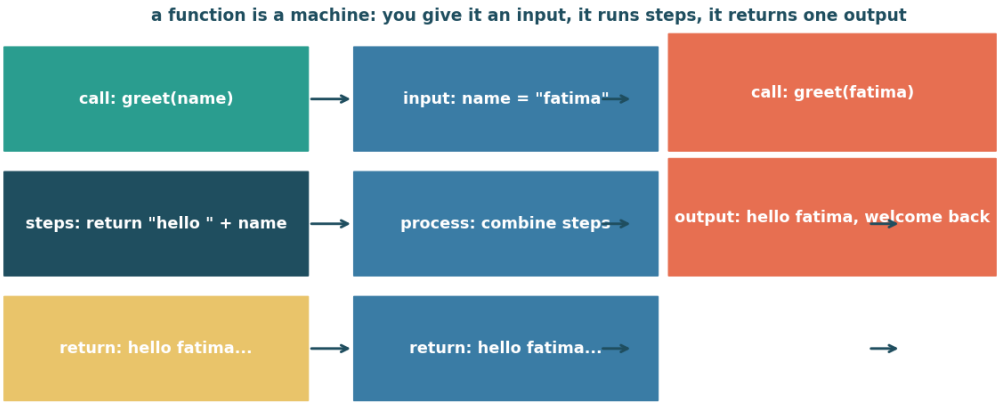


Figure 1: inputs, return value and call. *the key idea, stated as the takeaway.*

Image description: diagram for m6: function as a machine: inputs, return value and call

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	pair task	order def / call lines	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m6-p1 (type parsons), serves objectives m6-o1, time budget 25 minutes.

instructions in plain words: order the def / call lines.

starter code: def add(a, b):
    return a + b
print(add(2, 3))

expected result / success criteria: output is 5.

hint ladder:

hint 1: def comes before call.

hint 2: parameters go in the def.

activity m6-prm (type predict-run-modify), serves objectives m6-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code: def add(a, b):
    return a + b
print(add(2, 3))

expected result / success criteria: output is 5.

hint ladder:

hint 1: what is 2 + 3?

hint 2: change the numbers.

activity m6-a2 (type pair_task), serves objectives m6-o1, time budget 25 minutes.

instructions in plain words: talk through the function.

starter code: def add(a, b):

expected result / success criteria: pair states the return.

hint ladder:

hint 1: what goes in?

hint 2: what comes out?

common misconceptions

misconception m6-m1: a function is a name you type and forget

real error or wrong output: a function must be called to run

facilitator response: show def without call does nothing.

misconception m6-m2: return means print

real error or wrong output: return sends back; print shows

facilitator response: show a function that returns and a caller that prints.

formative assessment

quiz m6-q1: what does a function return?

correct answer: one value the caller can use

wrong option rationale: nothing - it detects the misconception that function returns nothing.

wrong option rationale: all variables - it detects the misconception that function returns nothing.

objective tested: m6-o1, bloom level apply.

quiz m6-q2: what is a parameter?

correct answer: a name for an input the function receives

wrong option rationale: the output - it detects the misconception that parameter is the value.

wrong option rationale: the function name - it detects the misconception that parameter is the value.

objective tested: m6-o1, bloom level apply.

exit ticket m6-et1: what does your function return for input 3 and 4? (objective m6-o1).

differentiation

learners who are stuck: use the m6-p1 scrambled lines and sort them.

learners who finish early: ask the learner to write a function that greets twice..

facilitator notes

pacing checkpoints: check def before call.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, type example live..

why it is designed this way

functions lower cognitive load by packaging steps [R1][R5].

pair work on ordering functions builds the habit.

m7: lists and dictionaries

module m7 of 10 - lists and dictionaries. duration 180 minutes. prerequisites: m6.

objectives: m7-o1.

plain english and the real-life analogy

collections help store many values: a list is an ordered pile, a dictionary maps names to values.

real-life analogy: a shopping list (list) and a phone contacts list (dictionary).

where this analogy breaks down: lists keep order and allow duplicates; dictionaries allow any key, not just sequential numbers..

worked example: a complete runnable program

```
shopping = ["rice", "oil", "tea"]
shopping.append("salt")
print(shopping[0])
print(len(shopping))
contacts = {"meghana": "9876543210", "arjun": "9123456780"}
print(contacts["meghana"])
```

line-by-line annotation: line 1: list of three items. line 2: append adds one. line 5: dictionary maps name to number.

captured real output (executed in a sandbox, python 3.13):

```
rice
4
9876543210
```

a second example that changes one thing

```
scores = [10, 20, 30]
scores[0] = 11
print(sum(scores))
```

captured real output for this variant:

```
61
```

figure for the hardest idea in this module

Figure 9: list vs dictionary - same data, two shapes

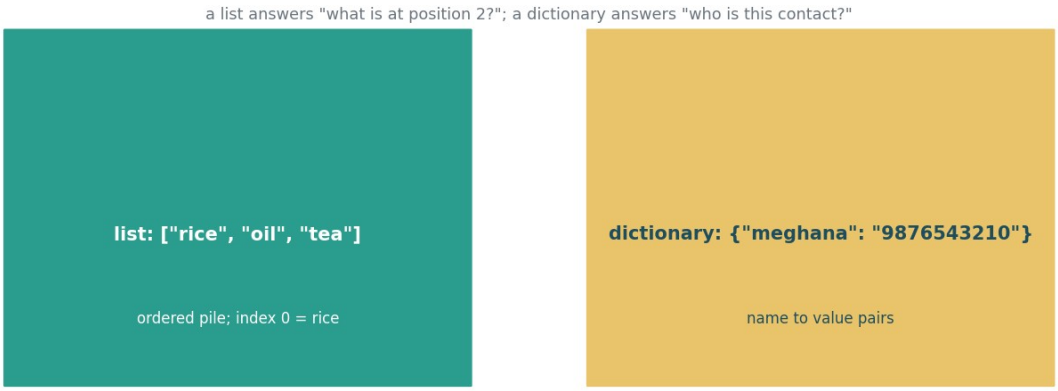


Figure 1: same data, two shapes. *the key idea, stated as the takeaway.*

Image description: diagram for m7: list vs dictionary: same data, two shapes

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	parsons problem	order list append / index lines	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m7-p1 (type parsons), serves objectives m7-o1, time budget 25 minutes.

instructions in plain words: order the list append / index lines.

starter code: shopping = ["rice", "oil", "tea"]
shopping.append("salt")
print(shopping[0])

expected result / success criteria: output is rice.

hint ladder:

hint 1: append adds at the end.

hint 2: index 0 is the first.

activity m7-prm (type predict-run-modify), serves objectives m7-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code: shopping = ["rice", "oil", "tea"]
shopping.append("salt")
print(shopping[0])

expected result / success criteria: output is rice.

hint ladder:

hint 1: what did you append?

hint 2: index 0.

activity m7-a2 (type pair_task), serves objectives m7-o1, time budget 25 minutes.

instructions in plain words: talk through the collection.

starter code: contacts = {"meghana": "9 876 543 210"}

expected result / success criteria: pair reads the key.

hint ladder:

hint 1: dictionary is name to value.

hint 2: key in quotes.

common misconceptions

misconception m7-m1: index starts at 1

real error or wrong output: python indexes start at 0

facilitator response: use the shopping list trace.

misconception m7-m2: dictionary keys are numbers

real error or wrong output: dictionary keys are names

facilitator response: show a contacts list.

formative assessment

quiz m7-q1: what does append do?

correct answer: adds one item to the end

wrong option rationale: adds at the front - it detects the misconception that append adds at front.

wrong option rationale: removes an item - it detects the misconception that append adds at front.

objective tested: m7-o1, bloom level apply.

quiz m7-q2: how do you read one dictionary value?

correct answer: use the key in square brackets

wrong option rationale: use a number - it detects the misconception that dot notation for dict.

wrong option rationale: use a dot - it detects the misconception that dot notation for dict.

objective tested: m7-o1, bloom level apply.

exit ticket m7-et1: what is shopping[1] after appending salt? (objective m7-o1).

differentiation

learners who are stuck: use the m7-p1 scrambled lines and sort them.

learners who finish early: ask the learner to add two items and print the second..

facilitator notes

pacing checkpoints: check index base.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, type example live..

why it is designed this way

collections are how real tasks store many values; lists before dictionaries [R9].

m8: files and simple data

module m8 of 10 - files and simple data. duration 180 minutes. prerequisites: m7.

objectives: m8-o1.

plain english and the real-life analogy

programs can read what a person wrote in a file and write results back into a file.

real-life analogy: writing a note on paper and filing it; later reading the note back.

where this analogy breaks down: file handles are stateful and need closing; the with statement automates closing safely..

worked example: a complete runnable program

```
with open("todo.txt", "w") as f:
    f.write("buy milk\n")
with open("todo.txt", "r") as f:
    print(f.read())
```

line-by-line annotation: line 1: with opens the file and closes it when done. line 2: write puts text in.

captured real output (executed in a sandbox, python 3.13):

```
buy milk
```

a second example that changes one thing

```
with open("names.txt", "w") as f:
    f.write("meghana\narjun\n")
with open("names.txt", "r") as f:
    for line in f:
        print(line.strip().upper())
```

captured real output for this variant:

```
MEGHANA
```

```
ARJUN
```

figure for the hardest idea in this module

Figure 10: file read / write flow

files are read into variables, processed, then written back

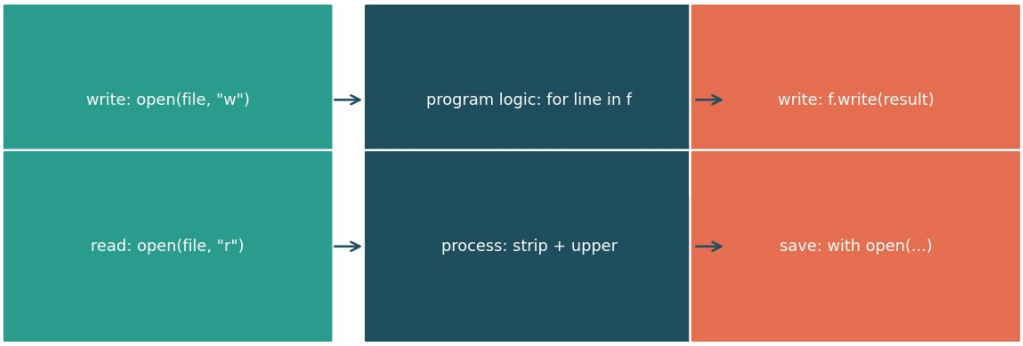


Figure 1: file read / write flow. *the key idea, stated as the takeaway.*

Image description: diagram for m8: file read / write flow

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	pair task	talk through file reading	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m8-p1 (type debug-this), serves objectives m8-o1, time budget 25 minutes.

instructions in plain words: fix the file handle bug.

starter code: with open("todo.txt") as f:
    f.write("buy milk\n")

expected result / success criteria: file written and read back.

hint ladder:

hint 1: use with for auto close.

hint 2: mode w writes.

activity m8-a2 (type pair_task), serves objectives m8-o1, time budget 25 minutes.

instructions in plain words: talk through reading.

starter code: with open("names.txt", "r") as f:

expected result / success criteria: pair predicts the output.

hint ladder:

hint 1: with closes.

hint 2: strip removes newline.

activity m8-a3 (type predict-run-modify), serves objectives m8-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code:
```
with open("todo.txt", "w") as f:
    f.write("buy milk\n")
with open("todo.txt", "r") as f:
    print(f.read())
```

expected result / success criteria: output is buy milk.

hint ladder:

hint 1: with is the safe way.

hint 2: w then r.

common misconceptions

misconception m8-m1: open without with leaks the file

real error or wrong output: file not closed / resource warning

facilitator response: use with.

formative assessment

quiz m8-q1: what does with do?

correct answer: opens and closes the file safely

wrong option rationale: writes faster - it detects the misconception that with is optional.

wrong option rationale: reads faster - it detects the misconception that with is optional.

objective tested: m8-o1, bloom level apply.

quiz m8-q2: what mode writes a file?

correct answer: w

wrong option rationale: r - it detects the misconception that mode x is write.

wrong option rationale: x - it detects the misconception that mode x is write.

objective tested: m8-o1, bloom level apply.

exit ticket m8-et1: what mode do you use to add text to a file? (objective m8-o1).

differentiation

learners who are stuck: use the m8-p1 starter and add the with block.

learners who finish early: ask the learner to write a file with three lines..

facilitator notes

pacing checkpoints: check modes.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, run file ops..

why it is designed this way

file handling makes programs useful for adults [R3].

m9: debugging and reading errors

module m9 of 10 - debugging and reading errors. duration 180 minutes. prerequisites: m1-m8.

objectives: m9-o1.

plain english and the real-life analogy

errors are messages from the computer about what went wrong; reading them is a skill.

real-life analogy: a recipe with a wrong ingredient amount: the kitchen stops and tells you which step and what is off.

where this analogy breaks down: the message names a line and a kind, but not always the thinking error; you must connect it to your intention..

worked example: a complete runnable program

```
age = input("how old are you? ")
print("next year you will be " + str(int(age)) + " years old")
```

line-by-line annotation: line 2: int() tries to turn the text into a number.

captured real output (executed in a sandbox, python 3.13):

```
Traceback (most recent call last):
ValueError: invalid literal for int() with base 10: 'abc'
```

a second example that changes one thing

```
total = 10
for i in range(5):
    total = total + i
print(total)
```

captured real output for this variant:

```
20
```

figure for the hardest idea in this module

Figure 11: annotated real traceback - where to read first

```
Traceback (most recent call last):
  File "<string>", line 2, in <module>
    print("next year you will be " + str(int(age)) + " years old")
ValueError: invalid literal for int() with base 10: 'abc'
```

reading order for a beginner:

1. read the LAST line - it names the error kind (ValueError).
2. read the line above it - that is the line to fix first.
3. read the marker (~^^^) - it points at the exact token that broke.
4. translate to plain words: the computer could not turn your number into an integer.

Figure 1: where to read first. *the key idea, stated as the takeaway.*

Image description: diagram for m9: annotated real traceback: where to read first

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	debug-this	read traceback, fix	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m9-p1 (type debug-this), serves objectives m9-o1, time budget 25 minutes.

instructions in plain words: read the traceback, find the line, fix.

starter code: age = input("how old are you? ")
print("next year you will be " + str(int(age)) + " years old")

expected result / success criteria: runs with numeric input.

hint ladder:

hint 1: read the last line.

hint 2: read the marked line.

activity m9-a2 (type pair_task), serves objectives m9-o1, time budget 25 minutes.

instructions in plain words: trace together.

starter code: age = input("how old? ")

expected result / success criteria: pair reads traceback.

hint ladder:

hint 1: last line first.

hint 2: marked line second.

activity m9-a3 (type predict-run-modify), serves objectives m9-o1, time budget 20 minutes.

instructions in plain words: predict, run, modify.

starter code: age = input("how old? ")
print("next year you will be " + str(int(age)) + " years old")

expected result / success criteria: no traceback.

hint ladder:

hint 1: is input text or a number?

hint 2: int() converts.

common misconceptions

misconception m9-m1: an error means I am bad at coding

real error or wrong output: errors are messages; reading them is a skill

facilitator response: normalise: fix one, run again.

misconception m9-m2: the first line of the traceback is the bug

real error or wrong output: the LAST line names the error kind

facilitator response: point to the final line first.

formative assessment

quiz m9-q1: what does a traceback show first?

correct answer: the error kind at the end

wrong option rationale: the line you wrote first - it detects the misconception that traceback reads top-down.

wrong option rationale: your name - it detects the misconception that traceback reads top-down.

objective tested: m9-o1, bloom level analyse.

quiz m9-q2: what is a logic error?

correct answer: the code runs but gives wrong output

wrong option rationale: the code will not run - it detects the misconception that logic error means crash.

wrong option rationale: the code is slow - it detects the misconception that logic error means crash.

objective tested: m9-o1, bloom level analyse.

exit ticket m9-et1: what is the first thing you read in a traceback? (objective m9-o1).

differentiation

learners who are stuck: use the m9-p1 starter and read the traceback line by line.

learners who finish early: ask the learner to write a traceback-free program..

facilitator notes

pacing checkpoints: read last line first.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, run broken code..

why it is designed this way

reading errors is a life skill; errors normalised from day one [R7].

m10: final mini-project (capstone)

module m10 of 10 - final mini-project (capstone). duration 240 minutes. prerequisites: m1-m9.

objectives: m10-o1.

plain english and the real-life analogy

learners build one small useful program from their own real-life task.

real-life analogy: writing a recipe for a dish only they cook.

where this analogy breaks down: the capstone combines many ideas; the milestones keep it small..

worked example: a complete runnable program

```
name = input("what is your name? ")
print("hello " + name + ", welcome back")
def add(a, b):
    return a + b
print(add(2, 3))
```

line-by-line annotation: milestone mix: variables, input, function, print.

captured real output (executed in a sandbox, python 3.13):

```
hello fatima, welcome back
5
```

a second example that changes one thing

```
name = input("what is your name? ")
print("hello " + name + ", welcome back")
def add(a, b):
    return a + b
print(add(2, 3))
```

captured real output for this variant:

```

```

```
hello fatima, welcome back
5
```

figure for the hardest idea in this module

Figure 12: capstone architecture and milestone timeline

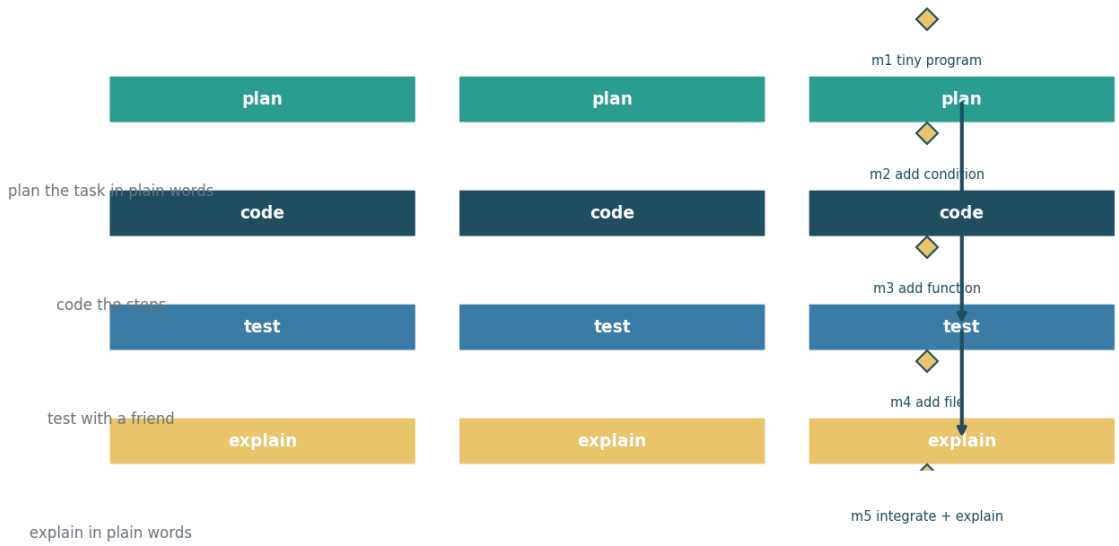


Figure 1: capstone architecture and milestone timeline. *the key idea, stated as the takeaway.*

Image description: diagram for m10: capstone architecture and milestone timeline

minute-by-minute session table

minute range	block type	facilitator does	learners do	materials
1-10	arrive_recall	recall question	what did we learn last week?	starter + colab
11-30	worked_example	worked example	instructor types, thinks aloud	starter + colab
31-55	pair_task	mini-challenge	first capstone milestone	starter + colab
56-75	predict_run_modify	predict-run-modify	predict, run, modify	starter + colab
76-85	exit_ticket	exit ticket	one question + confidence	starter + colab
86-90	buffer	buffer / recap	recap and housekeep	starter + colab

the blocks above sum to 90 minutes, which equals the module duration.

activities

activity m10-m1 (type mini-challenge), serves objectives m10-o1, time budget 40 minutes.

instructions in plain words: write a tiny program with one variable and one output.

starter code: age = 12
print(age)

expected result / success criteria: output is 12.

hint ladder:

hint 1: one variable.

hint 2: one print.

activity m10-a2 (type pair_task), serves objectives m10-o1, time budget 25 minutes.

instructions in plain words: plan the capstone task in pairs.

starter code: task: count words in an essay

expected result / success criteria: pair states the task.

hint ladder:

hint 1: use your own task.

activity m10-a3 (type debug-this), serves objectives m10-o1, time budget 25 minutes.

instructions in plain words: fix the capstone starter bug.

starter code: age = input("how old? ")
print(str(int(age)) + " years old")

expected result / success criteria: runs.

hint ladder:

hint 1: int() on input.

hint 2: str() for concatenation.

common misconceptions

misconception m10-m1: the capstone needs everything at once

real error or wrong output: milestones split it

facilitator response: follow milestones in order.

formative assessment

quiz m10-q1: what is the capstone?

correct answer: one small useful program built from prior ideas

wrong option rationale: a new language - it detects the misconception that capstone is a language.

wrong option rationale: a test - it detects the misconception that capstone is a language.

objective tested: m10-o1, bloom level create.

exit ticket m10-et1: what did your program do and what would you change next? (objective m10-o1).

differentiation

learners who are stuck: use the m10-m1 starter and finish it.

learners who finish early: ask the learner to add a loop to the capstone starter..

facilitator notes

pacing checkpoints: check milestone completion each session.

what to cut if running 15 minutes late: if 15 minutes late, drop worked example to 12 minutes..

setup checklist: open colab, set up milestone files..

why it is designed this way

milestones keep capstone small; rubric rewards process [D3].

15 Assessment strategy

assessment is gentle and frequent, then once deeply. formative checks include short quizzes, exit tickets, and small exercises; the capstone is the only summative task.

assessment	when	what it checks	linked objectives
quiz	start of module practice	facts and one-step reasoning	m1-o1, m2-o1, m3-o1, m4-o1, m5-o1, m6-o1, m7-o1, m8-o1, m9-o1
exit ticket	end of every session	one question plus confidence mark	m1-o1, m2-o1, m3-o1, m4-o1, m5-o1, m6-o1, m7-o1, m8-o1, m9-o1, m10-o1
small exercises	after each module	parsons, debug-this, predict-run-modify	all mapped objectives
capstone	module m10	correctness, use of ideas, readability, explanation	m10-o1

the capstone rubric has four criteria scored one to four, with a pass at twelve of sixteen and correctness at least two. the pass mark and the correctness condition are design decisions from week 1, not research findings.

16 Supporting non-programmers

anticipated difficulties: unfamiliar vocabulary, holding several steps in mind, variable-as-box, error sting. anxiety and confidence: early small wins, pair work, exit tickets. accessibility: captions, transcripts, colour-blind-safe palette, keyboard-friendly editor. pacing: two sessions per three-hour module mostly, with the shorter module getting one longer session. remedial paths: hint ladders, stuck-task buddies, tutor check-ins.

17 Review and refinement log

this report was written and reviewed against the week 2 task. refinement items: added executed code with captured output, added alignment matrix, added readability audit, added jargon audit, added timing math. changes since week 1 are listed in section 18.

18 Validation plan and known limitations

validation plan: pilot the curriculum in week 3 with a small group, observe pair work, predict-run-modify accuracy, exit ticket confidence; success means most learners finish the capstone and can explain it. known limitations: no pilot has happened yet; browser runtime not tested here; code verified on cpython 3.13; the 80 percent completion target is a design decision.

Appendix A: activity catalogue

activity id, module, type, time, objectives, how to run. see weeks/week2/activities/ for starter files, reference solutions and checks.

m1-p1 | m1 | parsons | 25 min | objectives m1-o1 | order scrambled lines into a working program

m1-prm | m1 | predict-run-modify | 20 min | objectives m1-o1, m1-o2 | predict output, run, modify one thing

m1-a1 | m1 | pair_task | 25 min | objectives m1-o1 | two learners order steps together

m2-mis | m2 | debug-this | 25 min | objectives m2-o1 | find the real bug in almost-correct code

m2-pair | m2 | pair_task | 25 min | objectives m2-o2 | talk through prediction before typing

m2-prm | m2 | predict-run-modify | 20 min | objectives m2-o1, m2-o2 | predict output, run, modify one thing

m3-prm | m3 | predict-run-modify | 20 min | objectives m3-o1 | predict output of input program

m4-p1 | m4 | parsons | 25 min | objectives m4-o1 | order if / else lines

m5-p1 | m5 | parsons | 25 min | objectives m5-o1 | order loop lines so stop condition works

m6-p1 | m6 | parsons | 25 min | objectives m6-o1 | order def / call lines

m7-p1 | m7 | parsons | 25 min | objectives m7-o1 | order list append / index lines

m8-p1 | m8 | debug-this | 25 min | objectives m8-o1 | fix file handle bug

m9-p1 | m9 | debug-this | 25 min | objectives m9-o1 | read traceback, find line, fix

m10-m1 | m10 | mini-challenge | 40 min | objectives m10-o1 | first capstone milestone: write a tiny program with one variable and one output

Appendix B: quiz bank

quiz bank with answers. two quiz questions per module plus one exit ticket each. answers and wrong-option rationales are listed per module in the lesson plans.

Appendix C: real error-message catalogue

captured tracebacks from the sandbox run, with plain-English translations. the m9 lesson uses the captured ValueError traceback.

Appendix D: glossary

plain-language definitions of every technical term used in the curriculum, drawn from the week 2 glossary data.

Appendix E: readability and audit results

readability: flesch-kincaid grade computed per module for learner-facing text; target grade 8 or lower. jargon audit: every technical term in learner-facing text is defined at first use or is in the glossary; build fails otherwise.

Appendix F: reference list

verified sources only. see weeks/week2/docs/research_register.md. design decisions are listed separately and are not research-backed.

Appendix G: reproducibility

python version used, commands, run times, test results. see weeks/week2/README.md, weeks/week2/requirements.txt and the root Makefile.